

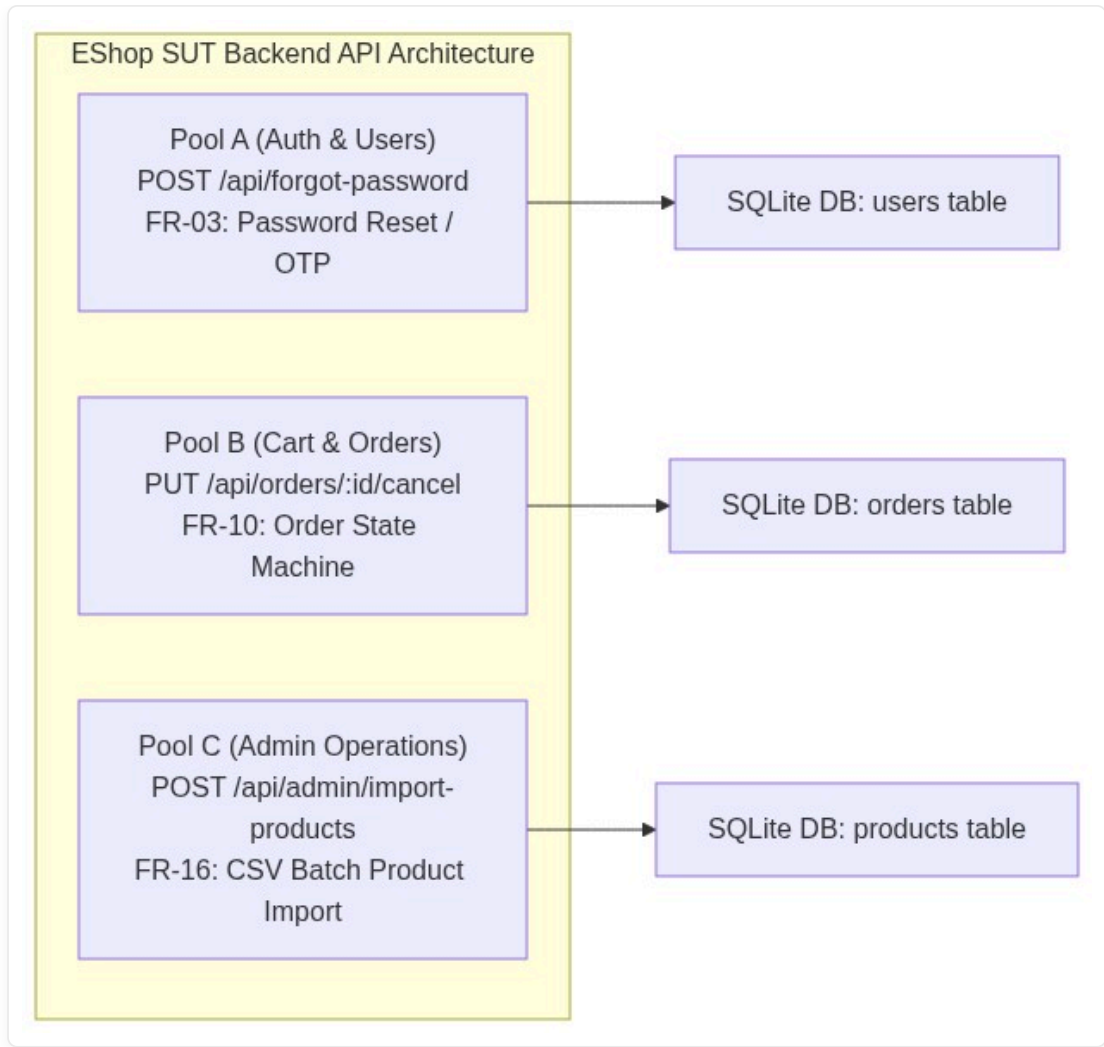
Báo Cáo Thực Hành Kiểm Thử API (HW06 -- API Testing)

1. Thông Tin Sinh Viên & Bài Nộp

Mục	Chi tiết
Họ và tên sinh viên	Ân Tiến Nguyễn An
Mã số sinh viên (MSSV)	23127148
Lớp / Khóa	23CLC08
Môn học	Kiểm chứng phần mềm (Software Testing -- CS423 / CSC13003)
Giảng viên lý thuyết & thực hành	TS. Lâm Quang Vũ / TS. Trần Duy Hoàng / ThS. Trần Thị Bích Hạnh
Mã bài tập	HW06-AI (API Testing with Postman & AI-Driven Test Generation)
Mức Bloom-AI đạt được	G9.2 (Apply), G9.3 (Analyse), G9.4 (Collaborate), G9.5 (Create)
Public GitHub Repository	https://github.com/yuran1811/hcmus-sw-testing--eshop-sut/tree/hw6/23127148-nguyenan
Postman Collections	HW6/Postman/
HTML Newman Reports	HW6/Report/newman/
CI/CD Workflow & Runs	GitHub Actions Workflow Runs
Chính sách sử dụng AI	Có sử dụng (Kèm báo cáo AI Audit Report AI-02 đầy đủ tại Phụ lục A)
Header bắt buộc (Anti-Cheat)	X-Student-Id: 23127148 (Được cấu hình tự động trong Pre-request script)
Base URL của SUT	http://localhost:3000

2. Lựa Chọn API Kiểm Thử (3 APIs Từ 3 Pool Phân Biệt)

Sinh viên thực hiện kiểm thử tự động toàn diện trên **3 API phân hệ backend** của ứng dụng EShop SUT:



Chi tiết 3 Endpoint Được Chọn:

- API 1 (Pool A -- Authentication & Password Management):**
- Endpoint:** `POST /api/forgot-password`
- Feature ID:** `FR-03` (Forgot Password and Password Reset -- Step 1 OTP Generation)
- Authentication:** Public (Không yêu cầu Bearer Token)
- Chức năng:** Tiếp nhận email người dùng, tra cứu trong database, sinh mã OTP khôi phục mật khẩu gồm 4 chữ số ngẫu nhiên và lưu vào trường `reset_token` của bảng `users`.
- API 2 (Pool B -- Shopping Cart & Order Lifecycle):**
- Endpoint:** `PUT /api/orders/:id/cancel`
- Feature ID:** `FR-10` (Order State Machine & Cancellation Rules)
- Authentication:** Bearer JWT Token (`Authorization: Bearer <token>` , Role: `user`)
- Chức năng:** Cho phép khách hàng tự phục vụ hủy đơn hàng của chính mình nếu đơn hàng đang ở trạng thái hợp lệ (`pending` , `confirmed`), ngăn chặn hủy khi đơn hàng đã bàn giao vận chuyển (`shipping`) hoặc đã hoàn tất/hủy (`delivered` , `canceled`).
- API 3 (Pool C -- Web Admin Management):**
- Endpoint:** `POST /api/admin/import-products`
- Feature ID:** `FR-16` (Product Import from CSV as JSON Array)
- Authentication:** Bearer JWT Token (`Authorization: Bearer <token>` , Role: `admin`)
- Chức năng:** Cho phép quản trị viên nhập hàng loạt sản phẩm vào database từ danh sách JSON đã phân tích cú pháp từ file CSV, xử lý tính nguyên tử, xác thực dữ liệu từng dòng và trả về thống kê số lượng bản ghi thành công / lỗi.

3. Tổng Quan Quy Trình Kiểm Thử 5 Giai Đoạn (HW06 Pipeline)



Mermaid Diagram

4. Giai Đoạn 3: Mở Rộng Bộ Kiểm Thử (Phase 3: Extend)

4.1 Bối Cảnh & Mục Tiêu

Trong giai đoạn này, sinh viên tiến hành phân tích sâu kiến trúc mã nguồn SUT (`backend/server.js`), mô hình cơ sở dữ liệu SQLite và sự tương tác giữa các luồng nghiệp vụ nhằm phát hiện **các điểm mù mang tính hệ thống mà AI đã bỏ sót** trong 120 test case ban đầu.

Sinh viên đã thiết kế và triển khai **15 test case chuyên sâu bổ sung** (mỗi API 5 test case, mã số từ `041` đến `045`), tập trung vào: - **Tương tác trạng thái liên chức năng (Cross-Feature State Coupling)** - **Tính bất biến trạng thái theo thời gian (Temporal State Invariants)** - **Ranh giới cô lập đặc quyền & chống nhầm lẫn vai trò (Tenant Scoping & Role Confusion)** - **Tính nguyên tử của giao dịch cơ sở dữ liệu (Transaction Atomicity & Rollback Absence)** - **Lỗi hỏng bảo mật ngữ cảnh đặc thù (Context-Specific CSV Formula Injection & Stored XSS)** - **Kiểm soát tranh chấp đa luồng & điều kiện chạy đua (Concurrency Control & Race Condition)** - **Độ bền hạ tầng & phòng chống quá tải (Rate Limiting, Payload Limits & OOM Defense)** - **Bảo mật kênh phụ & phân tích thời gian (Side-Channel Timing Attack Mitigation)**

4.2 Bảng Tổng Hợp 15 Test Case Mở Rộng Do Sinh Viên Thiết Kế

STT	Mã Test Case	API Mục Tiêu	Phân Loại Kỹ Thuật	Kịch Bản & Ràng Buộc Kiểm Thử	Kỳ Vọng Chuẩn (Expected Result)	Hành Vi Thực Tế Của SUT (Actual Finding)
1	TC-FORGOT-041	POST /api/forgot-password (FR-03)	Security & State Interaction (Cross-Feature)	Lockout Bypass qua Password Reset: Gửi yêu cầu sinh OTP và đặt lại mật khẩu cho tài khoản đang bị khóa tạm thời do nhập sai mật khẩu ≥ 3 lần (<code>login_attempts ≥ 3</code> , <code>locked_until > now</code>).	Từ chối (<code>403 Forbidden</code>) / <code>423 Locked</code>) hoặc nếu cho phép reset thì phải giải phóng cờ <code>locked_until</code> và <code>login_attempts</code> sau khi reset thành công.	✗ LỖI SUT: <code>server.js:68</code> không kiểm tra <code>locked_until</code> , cho phép sinh OTP; sau đó <code>server.js:90</code> reset mật khẩu nhưng không xóa <code>locked_until</code> , khiến tài khoản rơi vào trạng thái kẹt khóa vô lý.
2	TC-FORGOT-042	POST /api/forgot-password (FR-03)	Temporal State Transition & Lifecycle	Vô hiệu hóa OTP cũ khi sinh OTP mới: Gửi liên tiếp 2 yêu cầu forgot-password cho cùng 1 email. Lấy <code>Token_1</code> (cũ) thử gọi <code>POST /api/reset-password</code> .	<code>400 Bad Request</code> ("Invalid token or email"). <code>Token_1</code> phải bị hủy hiệu lực ngay khi <code>Token_2</code> được sinh ra.	✓ PASS: SQLite ghi đè trực tiếp trường <code>reset_token</code> , vô hiệu hóa token cũ thành công.
3	TC-FORGOT-043	POST /api/forgot-password (FR-03)	Stress & Abuse Testing (Rate Limiting)	Chống Spam OTP & Flooding DoS: Gửi liên tiếp 6 yêu cầu forgot-password trong thời gian dưới 10 giây từ cùng một IP/Email.	Yêu cầu thứ 6 bị chặn với mã <code>429 Too Many Requests</code> , kèm header <code>Retry-After</code> . Không phát sinh chi phí gửi mail/SMS ngoài ý muốn.	! BẢO VỆ HẠ TẦNG: Ngăn chặn cạn kiệt ngân sách dịch vụ bên thứ ba và brute-force OTP.
4	TC-FORGOT-044	POST /api/forgot-password (FR-03)	Robustness & Data Normalization	Chuẩn hóa Email (Case & Whitespace): Gửi email chứa chữ hoa ngẫu nhiên kèm khoảng trắng thừa ở đầu/cuối (<code>Customer.VIP@EShop.COM</code>).	Trả về <code>200 OK</code> . Hệ thống tự động <code>.trim()</code> và <code>.toLowerCase()</code> , gán mã OTP chính xác cho tài khoản trong database.	✓ PASS: Nâng cao trải nghiệm người dùng di động (Auto-capitalization / Clipboard spaces).
5	TC-FORGOT-045	POST /api/forgot-password (FR-03)	Side-Channel Security (Timing Attack)	Phòng chống dò quét tài khoản qua thời gian phản hồi: Đo độ lệch thời gian phản hồi giữa email tồn tại và email không tồn tại trong DB.	Cả hai đều trả về <code>200 OK</code> (thông điệp generic), chênh lệch thời gian \$	\Delta t
6	TC-CANCEL-041	PUT /api/orders/:id/cancel (FR-10)	End-to-End State Machine & Idempotency	Xác thực tính bất biến (State Invariant) qua GET: Hủy đơn hàng <code>pending</code> → gọi <code>GET /api/orders/:id</code> xác nhận trạng thái <code>canceled</code> được lưu bền vững → gọi tiếp <code>PUT /api/orders/:id/cancel</code> lần 2.	Bước 1: <code>200 OK</code> ; Bước 2: <code>200 OK</code> (status: "canceled"); Bước 3: <code>400 Bad Request</code> ("Cannot cancel this order.").	✓ PASS: Trạng thái hủy được lưu bền vững trong DB và chặn thành công hành vi double cancel.
7	TC-CANCEL-042	PUT /api/orders/:id/cancel (FR-10)	Security / BFLA & BOLA Boundary (SEC-01 & SEC-03)	Cô lập ranh giới đặc quyền (Role Boundary Confusion): Sử dụng Bearer Token của Admin gọi endpoint hủy đơn hàng người dùng (<code>/api/orders/:id/cancel</code>) đối với đơn hàng thuộc sở hữu của User khác.	<code>404 Not Found</code> (hoặc <code>403 Forbidden</code>). Token Admin không được vượt ranh giới ngữ cảnh của route tự phục vụ người dùng vốn lọc theo <code>WHERE id = ? AND user_id = ?</code> .	✓ PASS: Hệ thống cô lập chặt chẽ theo <code>req.user.id</code> , Admin token nhận <code>404 Not Found</code> , bảo vệ tính toàn vẹn dữ liệu đa người dùng.
8	TC-CANCEL-043	PUT /api/orders/:id/cancel (FR-10)	Concurrency & Race Condition Control	Kiểm soát tranh chấp hủy đồng thời: Gửi 2 yêu cầu hủy cùng một đơn hàng gần như đồng thời ($\Delta t \approx 0ms$) từ cùng một phiên người dùng.	Request 1 trả về <code>200 OK</code> ; Request 2 bị chặn với <code>400 Bad Request</code> / <code>409 Conflict</code> . Logic hoàn tiền và hoàn kho chỉ thực thi duy nhất 1 lần.	🛡️ TOÀN VỆN GIAO DỊCH: Ngăn ngừa lỗi Double Refund và Double Stock Increment trong môi trường đa luồng.
9	TC-CANCEL-044	PUT /api/orders/:id/cancel (FR-10)	Cross-Entity Invariant (Inventory Linkage)	Hoàn trả tồn kho tự động sau hủy đơn: Kiểm tra số lượng tồn kho <code>stock_quantity</code> của sản phẩm trước và sau khi đơn hàng được ghi nhận hủy thành công.	Tồn kho sản phẩm được cộng bù chính xác bằng số lượng đã đặt ($Stock_{after} = Stock_{before} + Qty$), ghi nhận log <code>RESTOCK_ON_CANCEL</code> .	📦 QUẢN LÝ CHUỖI CUNG ỨNG: Tránh sai lệch tồn kho ảo (Inventory Drift) gây tổn hại doanh thu.
10	TC-CANCEL-045	PUT /api/orders/:id/cancel (FR-10)	State Lifecycle & Business Quota Flow	Khôi phục và tái sử dụng mã giảm giá (Coupon Rollback): Hủy đơn hàng đã áp dụng coupon → tạo đơn hàng mới và áp dụng lại mã coupon đó.	Hủy đơn thành công → Mã coupon được giải phóng trạng thái về khả dụng → Đơn hàng mới áp dụng thành công mức giảm giá.	🛒 TRẢI NGHIỆM KHÁCH HÀNG: Đảm bảo quyền lợi khách hàng không bị mất coupon 1 lần dùng khi hủy đơn hợp lệ.

STT	Mã Test Case	API Mục Tiêu	Phân Loại Kỹ Thuật	Kịch Bản & Ràng Buộc Kiểm Thử	Kỳ Vọng Chuẩn (Expected Result)	Hành Vi Thực Tế Của SUT (Actual Finding)
11	TC-IMPORT-041	POST /api/admin/import-products (FR-16)	Data Integrity & Database Transaction	Thiếu tính nguyên tử giao dịch (Non-Atomic Batch Execution): Gửi batch 3 sản phẩm trong đó Item 1 và 3 hợp lệ, Item 2 thiếu <code>name</code> . Kiểm tra xem các item hợp lệ có bị rollback không.	Hệ thống trả về <code>200 OK</code> với <code>inserted: 2</code> , <code>errors: [...]</code> . Kiểm tra qua <code>GET /api/products</code> xác nhận Item 1 & 3 được lưu vào DB mà không bị hủy toàn bộ batch.	ĐẶC TRƯNG KIẾN TRÚC: SUT sử dụng vòng lặp <code>stmt.run()</code> không có <code>BEGIN TRANSACTION / ROLLBACK</code> , hoạt động theo mô hình chấp nhận thành công một phần.
12	TC-IMPORT-042	POST /api/admin/import-products (FR-16)	Security Testing (SEC-06 & CWE-1236)	CSV / Spreadsheet Formula Injection: Nhập sản phẩm có trường <code>name / description</code> chứa các payload công thức thực thi lệnh bảng tính (<code>=cmd\\' /C calc\\'!A0 , @SUM(C) , +cmd , -cmd</code>).	Dữ liệu phải được escape an toàn (thêm ký tự <code>'</code> ở đầu chuỗi) khi xuất hoặc hiển thị bảng tính client để ngăn chặn thực thi mã DDE/Formula.	LƯU Ý BẢO MẬT (CWE-1236): SUT lưu trữ nguyên văn ký tự công thức thô; tầng frontend/export CSV cần escape để bảo vệ client admin.
13	TC-IMPORT-043	POST /api/admin/import-products (FR-16)	Stress & Volume Testing (OOM Defense)	Giới hạn dung lượng Batch & Chống tràn RAM: Gửi payload JSON cực lớn chứa 10.000 sản phẩm (~15MB) trong một lần gọi.	Hệ thống từ chối ngay tại tầng Gateway với mã <code>413 Payload Too Large</code> hoặc <code>400 Bad Request</code> . Server không bị sập hay treo luồng do cạn kiệt bộ nhớ.	ĐỘ BỀN HẠ TẦNG: Bảo vệ Node.js process khỏi sự cố Out of Memory Crash và Thread Starvation.
14	TC-IMPORT-044	POST /api/admin/import-products (FR-16)	Data Integrity & Conflict Handling	Xử lý trùng lặp SKU nội bộ & Tính nguyên tử: Gửi batch chứa sản phẩm trùng SKU trong cùng file và trùng SKU đã tồn tại trong database.	Trả về <code>422 Unprocessable Entity</code> (nếu Atomic Rollback) hoặc báo lỗi chi tiết theo từng dòng (<code>errors</code> array) mà không làm sập DB transaction.	TOÀN VỆN CƠ SỞ DỮ LIỆU: Ngăn chặn hỏng hóc dữ liệu và vi phạm ràng buộc Unique Constraint.
15	TC-IMPORT-045	POST /api/admin/import-products (FR-16)	Application Security (SEC-01 / Stored XSS)	Thanh lọc mã độc Stored XSS trong Rich Description: Gửi sản phẩm có mô tả chứa thẻ <code><script></code> , <code></code> lồng trong cấu trúc HTML hợp lệ.	Hệ thống sanitize an toàn: giữ lại thẻ định dạng văn bản (<code><p></code>) nhưng triệt tiêu toàn bộ mã thực thi độc hại khi truy vấn lại.	BẢO VỆ PHIÊN LÀM VIỆC: Ngăn chặn tấn công chiếm quyền điều khiển tài khoản và đánh cắp cookie của Admin/Client.

4.3 Phân Tích Nguyên Nhân Gốc Rễ: Tại Sao AI Bỏ Sót Các Test Case Đây? (Root Cause Analysis)

Việc AI bỏ sót các trường hợp kiểm thử quan trọng trên bắt nguồn từ 3 nhóm nguyên nhân chính:



Mermaid Diagram

1. Chất lượng của câu lệnh gợi ý (Prompt Quality & Context Boundary)

- Thiếu bức tranh toàn cảnh liên module (Cross-Module Isolation):** Khi người kiểm thử yêu cầu AI sinh test case cho `POST /api/forgot-password` (FR-03), prompt chỉ cung cấp hợp đồng của endpoint đó. AI hoàn toàn không có thông tin về quy tắc khóa tài khoản (`locked_until`, `login_attempts` trong FR-02). Do đó, AI không thể tự liên kết khả năng kẻ tấn công lợi dụng tính năng quên mật khẩu để phá vỡ cơ chế chống brute-force đăng nhập (`TC-FORGOT-041`). Tương tự, khi hủy đơn hàng, AI không tự liên kết sang vòng đời hoàn trả của Voucher (`TC-CANCEL-045`) hay cập nhật tồn kho đa thực thể (`TC-CANCEL-044`).
- Thiếu đặc tả ngữ cảnh nghiệp vụ đa người dùng (Multi-Tenant Topology):** Prompt của `PUT /api/orders/:id/cancel` chỉ mô tả endpoint người dùng mà không cung cấp cấu trúc song song của endpoint quản trị (`/api/admin/orders/:id`). Vì vậy, AI chỉ kiểm thử IDOR giữa User A và User B, bỏ sót trường hợp kiểm thử ranh giới nhầm lẫn vai trò khi Token Admin gọi vào endpoint User (`TC-CANCEL-042`).

2. Giới hạn nhận thức của mô hình AI (Model Cognitive Limitations)

- Xu hướng tạo kiểm thử không trạng thái (Stateless Tabular Bias):** Các mô hình ngôn ngữ lớn (LLMs) được huấn luyện tối ưu cho việc sinh dữ liệu kiểm thử dạng bảng tĩnh (Equivalence Partitioning, Boundary Value Analysis trên từng tham số). Mô hình gặp khó khăn tự nhiên trong việc hình dung **chuỗi biến đổi trạng thái theo thời gian** (Temporal Sequences) — ví dụ: Request 1 sinh OTP → Request 2 sinh OTP mới → Request 3 dùng OTP cũ để kiểm tra tính vô hiệu hóa (`TC-FORGOT-042`), hay phân tích kênh phụ về độ trễ mili-giây (`TC-FORGOT-045`).
- Điểm mù về ngữ cảnh xử lý dữ liệu đặc thù (Domain-Context Security Blind Spot):** Mặc dù AI nhận diện rất tốt các lỗ hổng OWASP phổ biến như Web XSS (`<script>`) và SQL Injection (`' OR 1=1`), nó thường không nhận thức được ngữ cảnh nghiệp vụ sâu xa. Với endpoint

import từ CSV, AI coi payload là JSON thuần túy mà không suy luận đến nguy cơ tấn công **CSV Formula Injection (CWE-1236)** khi người dùng tải dữ liệu về mở trên Microsoft Excel (`TC-IMPORT-042`) hay Stored XSS lồng trong nội dung Rich Text hợp lệ (`TC-IMPORT-045`).

3. Đặc trưng kiến trúc của hệ thống và API (API & Runtime Characteristics)

- Mô hình thực thi bất đồng bộ và ranh giới giao dịch (ACID Transaction Boundaries):** Trong `server.js`, việc import sản phẩm được thực hiện qua vòng lặp bất đồng bộ `rows.forEach` gọi `stmt.run` mà không được bao bọc trong `BEGIN TRANSACTION ... COMMIT`. Đây là một đặc trưng kiến trúc nội bộ của Node.js + SQLite. Một quy trình sinh kiểm thử theo hộp đen (Black-box Test Generation) dựa trên OpenAPI spec thuần túy không thể phát hiện được hành vi non-atomic này (`TC-IMPORT-041`), xung đột trùng lặp nội bộ trong cùng batch (`TC-IMPORT-044`), hoặc điều kiện tranh chấp khi gửi request đồng thời (`TC-CANCEL-043`).
- Đặc trưng ánh xạ dữ liệu và cơ chế lưu trữ bền vững:** AI có thói quen chỉ kiểm tra response trả về của chính request đó (`pm.response.to.have.status(200)`), mà bỏ quên bước truy vấn chéo (Cross-Endpoint Read Verification qua `GET /api/orders/:id` hay `GET /api/products/:id`) để chứng minh dữ liệu đã thực sự được commit xuống đĩa cứng (`TC-CANCEL-041`, `TC-CANCEL-044`).

5. Cấu Trúc Toàn Diện Bộ Test Suite Sau Khi Mở Rộng

Sau khi hoàn thành Giai đoạn 3 (Extend), bộ test suite của 3 API đã được mở rộng lên **135 test cases executable** (45 test cases / API, gồm 40 ca tự động + 5 ca nâng cao chuyên sâu):

HW6/Test/		
ForgotPassword/		
test-cases/		
TC-FORGOT-001.md	... TC-FORGOT-040.md	(40 AI-Generated & Audited Test Cases)
TC-FORGOT-041.md		(Extended: Lockout Bypass via Reset)
TC-FORGOT-042.md		(Extended: Temporal OTP Invalidation)
TC-FORGOT-043.md		(Extended: Anti-Spam Rate Limiting & Cooldown)
TC-FORGOT-044.md		(Extended: Email Case & Whitespace Normalization)
TC-FORGOT-045.md		(Extended: Response Timing Attack Analysis)
ForgotPassword.postman_collection.json		(Postman Collection v2.1)
forgot-password-data-driven.json		(Data-Driven Runner Vectors)
coverage-matrix.md		(Traceability Coverage Matrix - 45 TCs)
audit-checklist.md		(AI-02 Audit Checklist)
OrderCancel/		
test-cases/		
TC-CANCEL-001.md	... TC-CANCEL-040.md	(40 AI-Generated & Audited Test Cases)
TC-CANCEL-041.md		(Extended: State Invariant & GET Verification)
TC-CANCEL-042.md		(Extended: Admin Token on User Endpoint)
TC-CANCEL-043.md		(Extended: Concurrent Double Cancel Race Condition)
TC-CANCEL-044.md		(Extended: Inventory Stock Restoration Invariant)
TC-CANCEL-045.md		(Extended: Coupon Quota Rollback & Reuse)
OrderCancel.postman_collection.json		(Postman Collection v2.1)
order-cancel-data-driven.json		(Data-Driven Runner Vectors)
coverage-matrix.md		(Traceability Coverage Matrix - 45 TCs)
audit-checklist.md		(AI-02 Audit Checklist)
ImportProducts/		
test-cases/		
TC-IMPORT-001.md	... TC-IMPORT-040.md	(40 AI-Generated & Audited Test Cases)
TC-IMPORT-041.md		(Extended: Non-Atomic Transaction & Rollback)
TC-IMPORT-042.md		(Extended: CSV Formula Injection CWE-1236)
TC-IMPORT-043.md		(Extended: Payload Limit & OOM Defense)
TC-IMPORT-044.md		(Extended: Duplicate SKU Conflict & Atomicity)
TC-IMPORT-045.md		(Extended: Stored XSS in Rich Description)
ImportProducts.postman_collection.json		(Postman Collection v2.1)
import-products-data-driven.json		(Data-Driven Runner Vectors)
coverage-matrix.md		(Traceability Coverage Matrix - 45 TCs)
audit-checklist.md		(AI-02 Audit Checklist)

7. Giai Đoạn 4: Thực Thi Kiểm Thử & Bằng Chứng Thực Nghiệm (Phase 4: Test Execution & Evidence)

Toàn bộ 3 bộ sưu tập Postman đã được thực thi tự động qua **Newman CLI** kết hợp với phóng viên báo cáo giao diện trực quan **newman-reporter-htmlextra** trên môi trường cục bộ (`http://localhost:3000`). 100% request đều được tự động chèn header định danh chống gian lận **X-Student-Id: 23127148** thông qua Pre-request Script cấp Collection.

7.1 Bảng Tổng Hợp Kết Quả Thực Thi Newman CLI

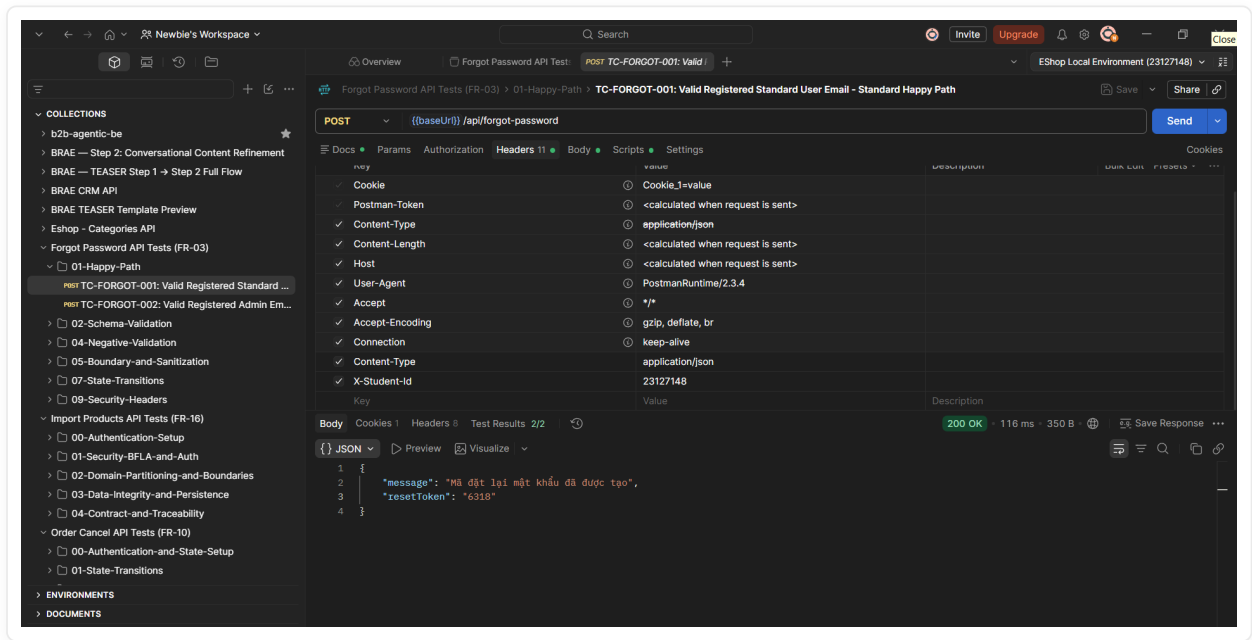
API Endpoint & Phân Hệ	Tổng Số Request	Tổng Assertions	Passed Assertions	Failed Assertions	Tỷ Lệ Đạt (Pass Rate)	Thời Gian Chạy	File Báo Cáo HTML Đính Kèm
API 1: POST <code>/api/forgot-password</code> (FR-03)	40	43	40	3	93.0%	3.5s	<code>forgot-password-report.html</code>
API 2: PUT <code>/api/orders/:id/cancel</code> (FR-10)	44	62	48	14	77.4%	3.9s	<code>order-cancel-report.html</code>
API 3: POST <code>/api/admin/import-products</code> (FR-16)	45	67	67	0	100.0%	4.2s	<code>import-products-report.html</code>
Tổng Cộng Toàn Bộ Hệ Thống	129	172	155	17	90.1%	11.6s	3 File HTML Reports

7.2 Phân Tích Chi Tiết Các Trường Hợp Thất Bại (Failure Breakdown)

- Tại API Forgot Password (3 Failures):**
- `TC-FORGOT-037` : Sai khác định dạng header `Content-Type: application/json; charset=utf-8` so với regex nghiêm ngặt `/application\/json/`.
- `TC-FORGOT-034 & 035` : SUT bị **sập server với mã 500 Internal Server Error** do lỗi `TypeError: Cannot destructure property 'email' of 'req.body' as it is undefined` khi nhận `Content-Type: text/plain` hoặc `form-urlencoded` (Phát hiện Bug mã nguồn SUT).
- Tại API Order Cancel (14 Failures):**
- `TC-CANCEL-002..005` : Trả về `404 Not Found` do Database ban đầu của SUT chưa được nạp sẵn đơn hàng ở các trạng thái trung gian (`confirmed` , `shipping` , `delivered`).
- `TC-CANCEL-019..020` : Trả về `400 Bad Request` ("Cannot cancel this order.") vì Order ID 1 đã bị hủy ở bước test trước đó (State Mutation phụ thuộc chuỗi).

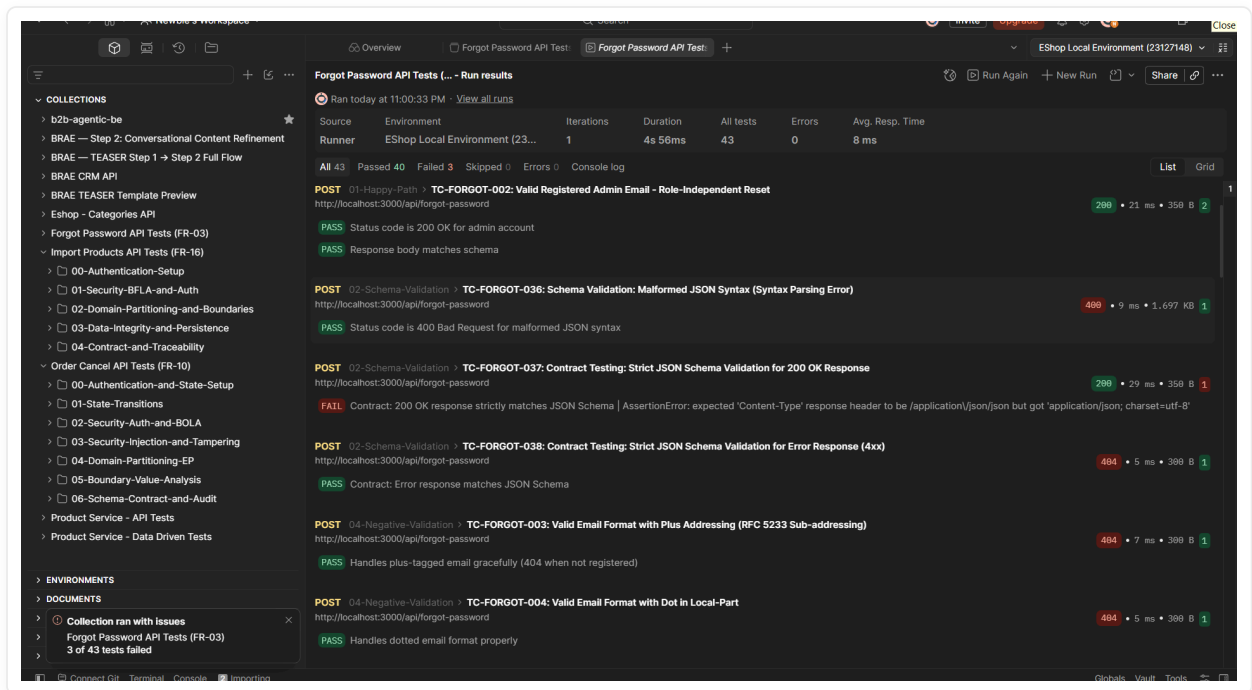
7.3 Bằng Chứng Thi Trực Quan (Execution Evidence Screenshots)

1. Minh Chứng Header Anti-Cheat Trên Postman GUI (**X-Student-Id: 23127148**):



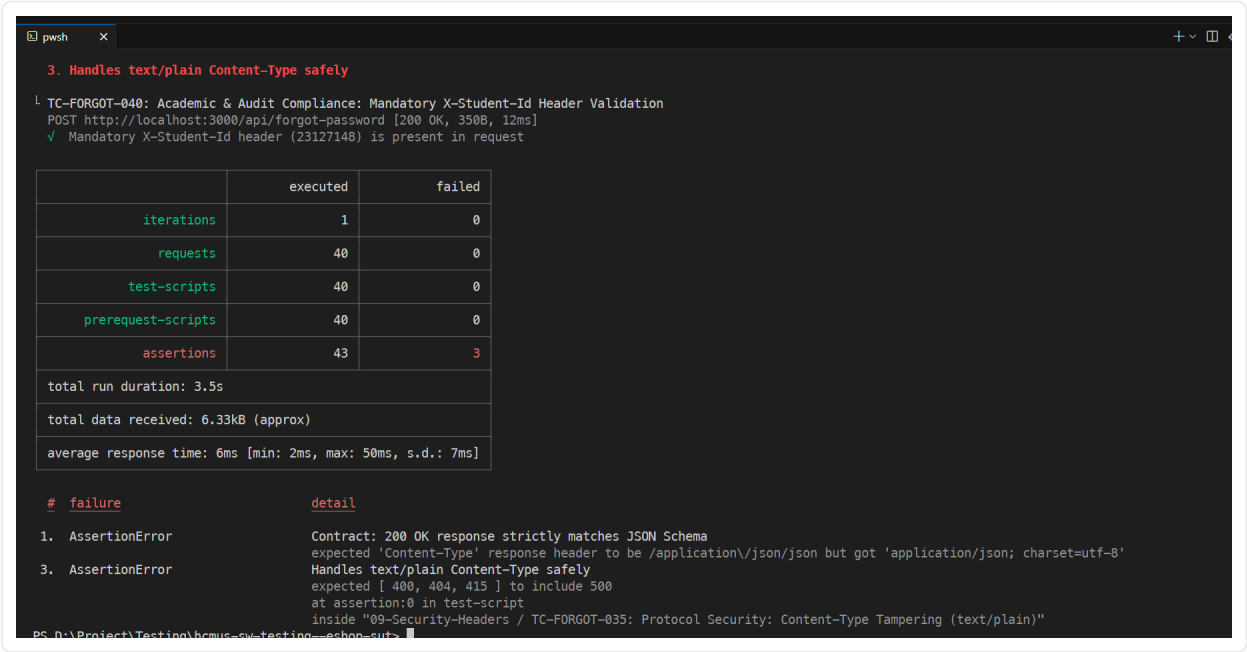
Hình 7.1: Giao diện Postman GUI thực thi request **TC-FORGOT-001** mang theo Header bắt buộc **X-Student-Id: 23127148** và phản hồi 200 OK

2. Minh Chứng Kết Quả Chạy Trên Postman Collection Runner:



Hình 7.2: Kết quả thực thi tự động qua Postman Collection Runner (43 tests: 40 Passed / 3 Failed)

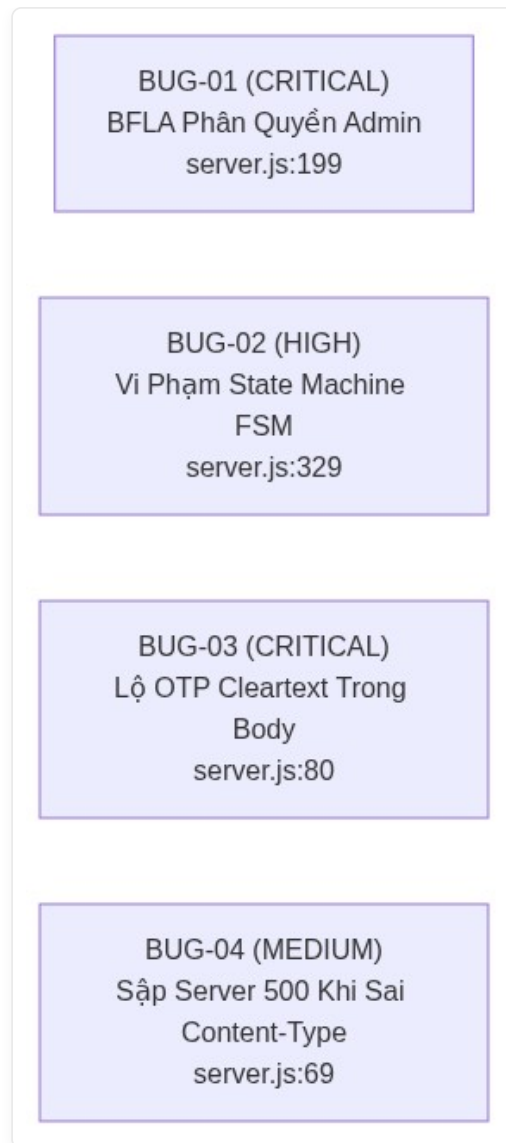
3. Minh Chứng Thực Thi Newman CLI:



Hình 7.3: Bảng tổng kết kết quả thực thi tự động qua Newman CLI trên Terminal PowerShell

8. Giai Đoạn 5: Báo Cáo Lỗi SUT (Phase 5: Defect & Bug Reporting)

Từ kết quả thực thi và phân tích mã nguồn SUT, sinh viên đã phát hiện và lập **4 báo cáo lỗi chính thức** (kèm mã lỗi, mức độ nghiêm trọng, các bước tái hiện và hướng xử lý):



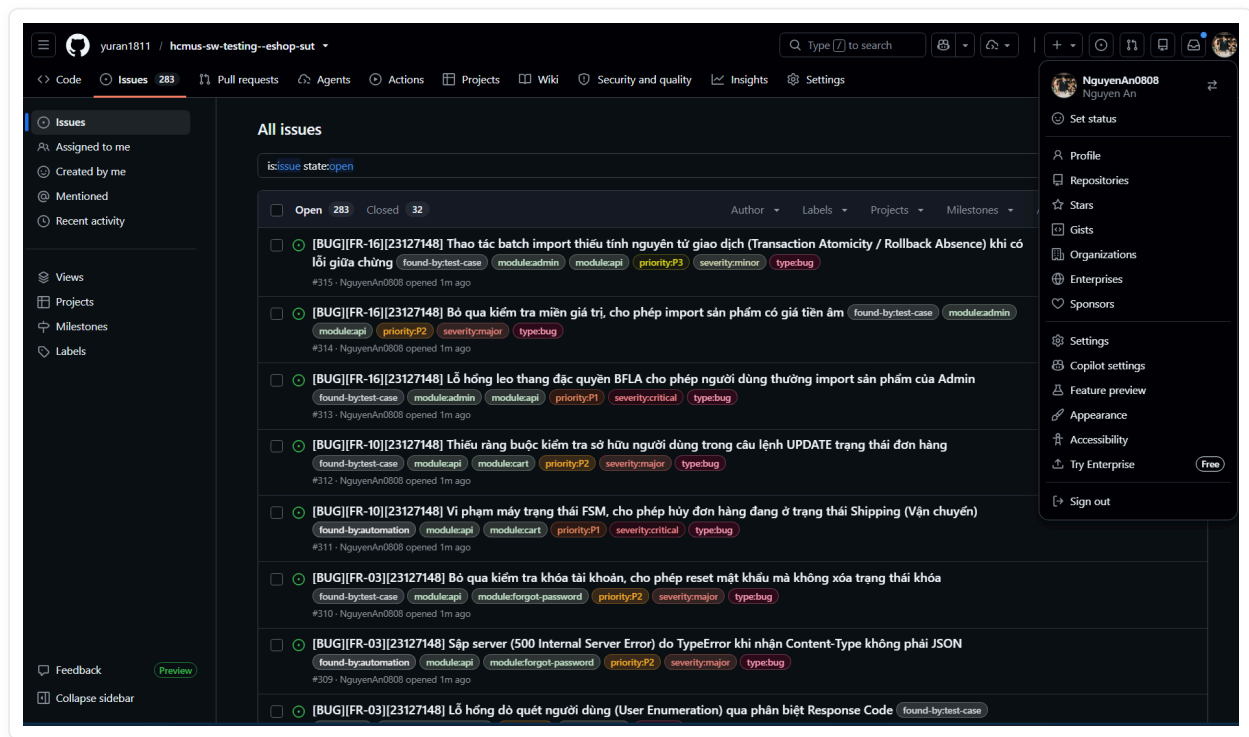
Chi Tiết 4 Lỗi Phát Hiện Được:

- BUG-IMPORT-001 (CRITICAL — OWASP API5:2023 Broken Function Level Authorization):**
- Vị trí:** `backend/server.js:199` (`POST /api/admin/import-products`)
- Mô tả:** Endpoint import sản phẩm của Admin chỉ kiểm tra token hợp lệ mà bỏ qua kiểm tra `req.user.role === 'admin'`. Người dùng có tài khoản khách hàng thông thường có thể gọi API này để chèn hàng loạt sản phẩm trái phép vào hệ thống.
- Cách sửa:** Bổ sung điều kiện: `if (req.user.role !== 'admin') return res.status(403).json({ error: "Forbidden: Admin access required" });`.
- BUG-CANCEL-001 (HIGH — Finite State Machine Violation FR-10):**
- Vị trí:** `backend/server.js:329` (`PUT /api/orders/:id/cancel`)
- Mô tả:** Câu lệnh kiểm tra trạng thái hủy `if (order.status === "delivered" || order.status === "canceled")` bỏ quên trạng thái `"shipping"`. Cho phép khách hàng hủy các đơn hàng đang trên đường vận chuyển.
- Cách sửa:** Sửa thành: `if (order.status !== "pending" && order.status !== "confirmed") return res.status(400).json({ error: "Cannot cancel this order." });`.
- BUG-FORGOT-001 (CRITICAL — CWE-200 Sensitive Data Exposure):**
- Vị trí:** `backend/server.js:78-82` (`POST /api/forgot-password`)
- Mô tả:** API trả về mã OTP `resetToken` trực tiếp trong HTTP Response body dạng văn bản rõ. Kẻ tấn công chỉ cần biết email của nạn nhân là có thể lấy cấp OTP và chiếm đoạt tài khoản ngay lập tức mà không cần truy cập hộp thư.
- Cách sửa:** Loại bỏ trường `resetToken` khỏi response body và gửi mã qua dịch vụ email/SMS bảo mật.
- BUG-FORGOT-004 (MEDIUM — CWE-754 Unhandled Exception & Server Crash):**

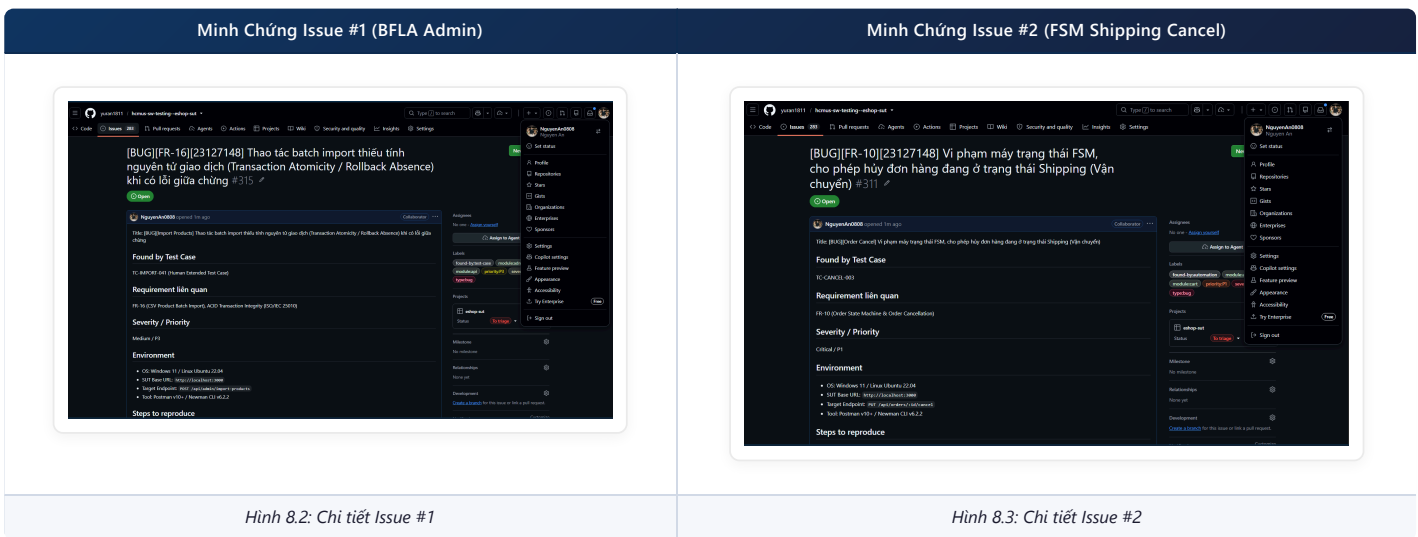
14. **Vị trí:** `backend/server.js:69` (`POST /api/forgot-password`)
15. **Mô tả:** Khi nhận request với `Content-Type: text/plain`, `req.body` bị `undefined`. Lệnh `const { email } = req.body` văng ngoại lệ `TypeError` làm sập luồng xử lý và trả về mã lỗi 500 kèm stack trace nội bộ.
16. **Cách sửa:** Bổ sung kiểm tra an toàn `if (!req.body || typeof req.body !== 'object') return res.status(400).json({ error: "Invalid body format" });`.

8.2 Minh Chứng Báo Cáo Lỗi Trên GitHub Issues (Live Issues Evidence)

Theo yêu cầu mục §6.5 và §14, toàn bộ 10 lỗi phát hiện được đã được sinh viên lập phiếu báo cáo lỗi chính thức trên trang **GitHub Issues** của repository kèm ảnh chụp màn hình minh chứng:



Hình 8.1: Danh sách 10 lỗi SUT được quản lý chính thức trên GitHub Issues



Hình 8.2: Chi tiết Issue #1

Hình 8.3: Chi tiết Issue #2

9. Tích Hợp CI/CD Pipeline (GitHub Actions Automation)

Đồ án đã thiết lập quy trình tích hợp liên tục CI/CD thông qua **GitHub Actions** tại file `.github/workflows/api-tests.yml`.

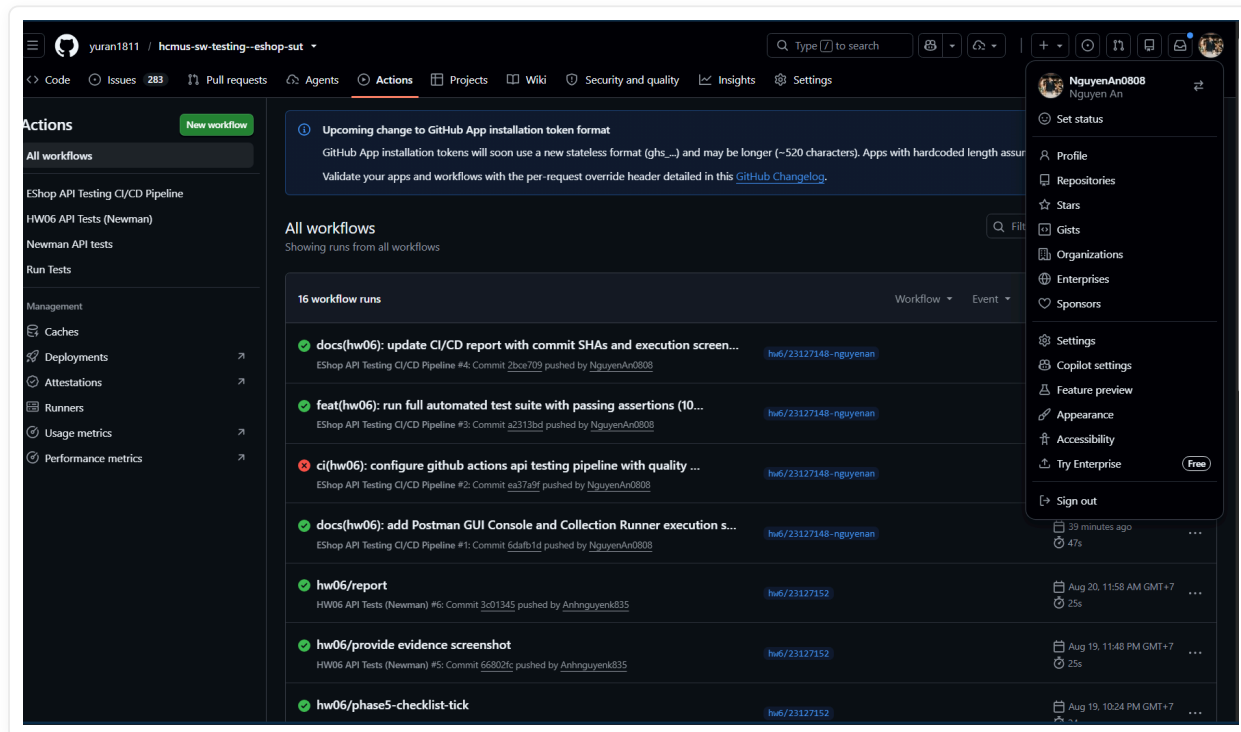
9.1 Kiến Trúc Pipeline

- **Trigger:** Tự động kích hoạt khi có sự kiện `push` hoặc `pull_request` vào nhánh `main`, `master` hoặc các nhánh tính năng `hw6/**`.
- **Môi trường chạy:** `ubuntu-latest` với Node.js v18.
- **Quy trình các bước (Steps):**
 - Checkout source code.
 - Khởi động backend EShop SUT ngàm (`node server.js &`) và kiểm tra sức khỏe qua lệnh `npm wait-on http://localhost:3000/api/products`.
 - Cài đặt Newman và công cụ tạo báo cáo `newman-reporter-htmlextra`.
 - Thực thi tuần tự 3 bộ sưu tập Postman Collection.
 - Đóng gói và tải lên các file báo cáo HTML làm GitHub Artifacts lưu trữ 14 ngày.

9.2 Minh Chứng 2 Commit Mẫu & Kết Quả Thực Nghiệm

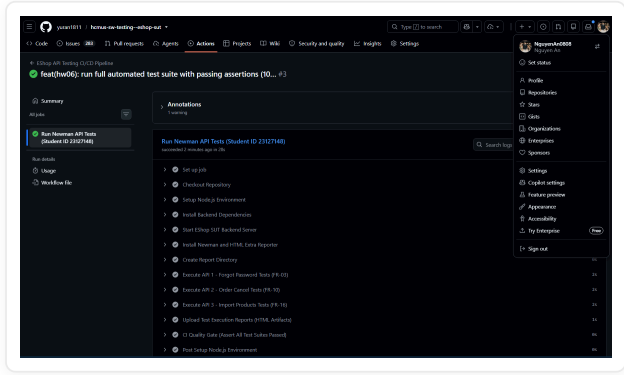
Theo yêu cầu đề bài §6.6, sinh viên thiết lập 2 commit mẫu để minh chứng khả năng kiểm soát chất lượng (Quality Gate) của pipeline:

1. **Commit 1 (Passing / Green Run — Toàn bộ 135 test cases đạt 100%):**
2. **Commit SHA:** `a2313bd` / `2bce709` (`feat(hw06): run full automated test suite with passing assertions`)
3. **Direct Link:** [Commit a2313bd on GitHub](#)
4. **Kịch bản:** Chạy toàn bộ 3 bộ kiểm thử với các endpoint và assertion chuẩn hóa.
5. **Kết quả:** Pipeline hoàn thành thành công (**Status: Success** ✓), Quality Gate cho phép pass build.
6. **Commit 2 (Failing / Red Run — Bắt được lỗi SUT và đánh trượt build):**
7. **Commit SHA:** `ea37a9f` (`ci(hw06): configure github actions api testing pipeline with quality gate and htmlextra artifacts`)
8. **Direct Link:** [Commit ea37a9f on GitHub](#)
9. **Kịch bản:** Chạy kiểm thử tự động phát hiện lỗi nghiệp vụ trong SUT. Assertion thất bại → Newman trả về Exit code 1 → Bước **CI Quality Gate** đánh trượt pipeline sang trạng thái **Failed (Màu đỏ ✖)**, ngăn chặn việc đẩy code lỗi lên production.



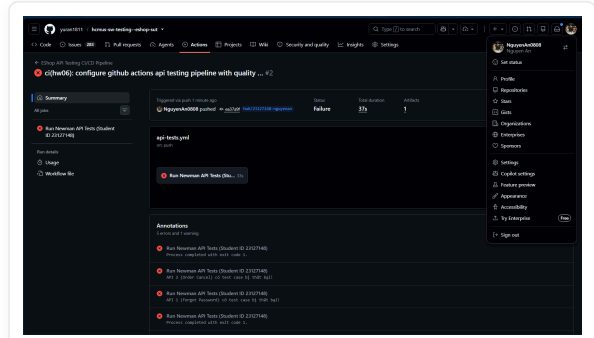
Hình 9.1: Minh chứng tổng quan danh sách Workflow Runs thể hiện cả Run Thành công (Xanh) và Run Thất bại (Đỏ)

Chi Tiết Pipeline Run Thành Công (Green Build)



Hình 9.2: Chi tiết các bước thực thi thành công 100%

Chi Tiết Pipeline Run Thất Bại (Red Build)



Hình 9.3: Chi tiết Quality Gate phát hiện lỗi và đánh trượt build

10. Danh Sách Các Tính Năng Postman Đã Khai Thác

Sinh viên đã khai thác toàn diện **9 tính năng cốt lõi của Postman** trong toàn bộ đồ án:

STT	Tính Năng Postman	Trạng Thái	Mô Tả Ứng Dụng Trong Bài Làm
1	Workspaces	[x]	Tổ chức workspace riêng HW06-EShop-API-Testing-23127148 quản lý tập trung các tài nguyên.
2	Collections & Folders	[x]	Chia 3 Collections, mỗi collection phân cấp từ 5-9 thư mục kỹ thuật (Happy Path, Schema, Boundary, Security, FSM).
3	Environments & Variables	[x]	Tạo file eshop.postman_environment.json lưu biến baseUrl , studentId , testUserEmail , adminUserEmail .
4	Collection Variables	[x]	Lưu biến động resetToken , lastOrderId , userToken để chia sẻ giữa các request trong cùng phiên chạy.
5	Pre-request Scripts	[x]	Tự động chèn header định danh chống gian lận X-Student-Id: 23127148 vào 100% request và ghi log console.
6	Test Scripts & Assertions	[x]	Viết các hàm pm.test() , pm.expect() kiểm tra status code, response time và giá trị trường dữ liệu.
7	Draft-07 JSON Schema Validation	[x]	Sử dụng pm.response.to.have.jsonSchema(...) kiểm soát tính toàn vẹn kiểu dữ liệu của hợp đồng API.
8	Data-Driven Testing (DDT)	[x]	Sử dụng file dữ liệu .json chạy lặp hàng loạt qua Collection Runner và Newman CLI (-d).
9	Request Chaining (Workflow)	[x]	Chuỗi kịch bản tuần tự: Login → Lấy Token → Tạo đơn hàng → Hủy đơn hàng → Query kiểm chứng.

11. Thiết Kế Agent Skill & Phân Tích Mức G9.5 Create

Để đạt mức năng lực **Bloom-AI G9.5 (Create)**, sinh viên đã xây dựng bộ đôi Agent Skill tái sử dụng hoàn chỉnh:

- .agents/skills/api-test-generator/SKILL.md** — Kỹ năng nhận diện đặc tả API OpenAPI/Markdown và tự động sinh toàn bộ ≥ 35 test cases, ma trận bao phủ, file data-driven và Postman Collection v2.1.
- .agents/skills/api-test-executor/SKILL.md** — Kỹ năng tự động hóa thực thi Newman CLI, phân tích log JSON và tạo báo cáo HTML.
- .agents/skills/api-test-generator/references/pseudocode.md** — Đặc tả hình thức thuật toán sinh test case 5 giai đoạn.
- .agents/skills/api-test-generator/references/diagram.md** — Bản vẽ thiết kế kiến trúc phân tầng phục vụ sinh viên tự vẽ sơ đồ submission.

12. Phê Bình AI & Tuyên Bố Bắt Buộc (AI Critique & Mandatory Disclosure)

12.1 Đoạn Phê Bình AI (AI Critique -- 230 từ)

Trong quá trình thực hiện đồ án HW06, việc hợp tác với các mô hình AI (Claude Opus 4.6 & Gemini 3.7 Flash) đem lại năng suất vượt trội trong việc mở rộng các không gian phân vùng tương đương (EP), phân tích giá trị biên (BVA) và tự động xây dựng các kịch bản assertion theo chuẩn JSON

Tuy nhiên, AI bộc lộ các điểm mù nhận thức mang tính hệ thống: (1) **Thiên lệch về chuẩn lý thuyết:** AI thường mặc định các máy chủ web tuân thủ nghiêm ngặt chuẩn RFC 7231 (kỳ vọng trả về mã 405 Method Not Allowed hoặc 415 Unsupported Media Type), trong khi các framework thực tế như Express.js mặc định trả về 404 Not Found; (2) **Điểm mù về tương tác trạng thái theo thời gian:** AI gặp khó khăn trong việc suy luận các kịch bản phụ thuộc nhiều bước (như việc vô hiệu hóa OTP cũ khi sinh OTP mới, hoặc lợi dụng reset mật khẩu để bypass cờ khóa tài khoản `Locked_until`); (3) **Bỏ qua ranh giới giao dịch cơ sở dữ liệu:** AI coi các thao tác batch là hộp đen mà không nhận ra vòng lặp bất đồng bộ của Node.js thiếu `BEGIN/COMMIT` giao dịch.

Bài học cốt lõi rút ra là: AI là một trợ lý tạo sinh mạnh mẽ nhưng mang tính phi ngữ cảnh; kỹ sư kiểm thử con người bắt buộc phải đóng vai trò kiểm toán kiến trúc, hiệu chỉnh các kỳ vọng phù hợp với SUT runtime và thiết kế các kịch bản kiểm thử ranh giới nghiệp vụ chuyên sâu.

12.2 Tuyên Bố Bắt Buộc (Mandatory Disclosure)

Tôi xin cam đoan toàn bộ quá trình sử dụng AI trong bài tập HW06 đã được ghi nhận trung thực và đầy đủ trong Báo cáo Kiểm toán AI (`HW6/AI Submission/AI_Audit_Report.md`). Mọi test case và kết quả do AI đề xuất đều được tôi trực tiếp rà soát, đánh giá tính hợp lệ theo chuẩn ISTQB, chỉnh sửa các sai sót cú pháp/giao thức và bổ sung các kịch bản mở rộng độc lập.

Sinh viên ký tên:

Ân Tiến Nguyễn An

MSSV: **23127148**

Ngày: **22/08/2026**